National University of Computer and Emerging Sciences



Laboratory Manuals

for

Data Structures Lab

(CL-2001)

Course Instructor	Mr. Waqas Rana
Lab Instructor	Ms. Fatima Ali
Lab TA	Mr. Abdur Rehman
Section	BSE-3B
Semester	Fall 2026

Department of Computer Science
FAST-NU, Lahore, Pakistan

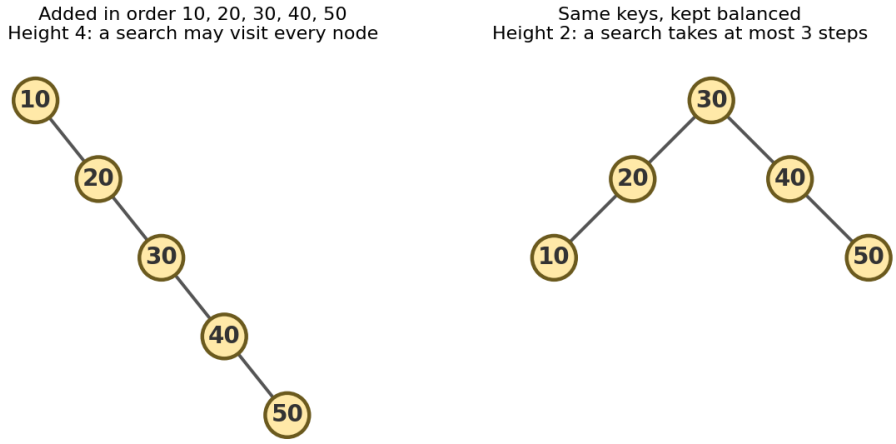
Lab Manual 08

Objectives:

- Understand why a plain BST can become slow, and how an AVL tree fixes that
- Calculate balance factors and choose the right rotation
- Build an AVL tree in C++ that balances itself after every insert and delete

Why do we need AVL trees?

A Binary Search Tree (BST) is fast when it is bushy. Finding a key takes about log n steps. But the shape of the tree depends on the order of the data. If you insert sorted keys like 10, 20, 30, 40, 50, every new key goes to the right and the tree becomes a straight line. Now a search may visit every node, which is no better than a linked list.



An AVL tree solves this problem. It checks itself after every insert and delete, and fixes its own shape when one side gets too tall.

What is an AVL tree?

An AVL tree is a BST with one extra rule:

- For every node, the heights of its left and right subtrees differ by **at most 1**.

It is named after its inventors, Adelson-Velsky and Landis (1962), and it was the first self-balancing BST.

Height and balance factor

The height of a node is the number of edges on the longest path from that node down to a leaf. That gives us three simple facts:

- A leaf has height **0**
- A missing child (nullptr) counts as height **-1**
- Height of a node = 1 + the larger of its two children's heights

The balance factor (BF) of a node tells us how lopsided it is:

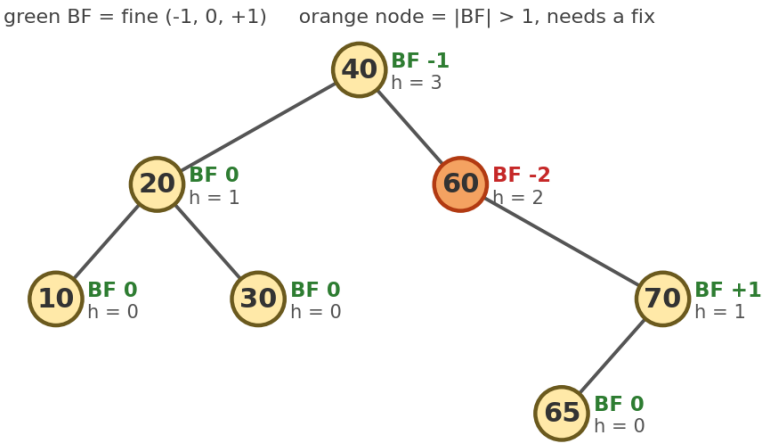
BF = height of left subtree - height of right subtree

BF	Meaning	Allowed?
+1	Left side is one level taller	Yes
0	Both sides are the same height	Yes
-1	Right side is one level taller	Yes

BF	Meaning	Allowed?
+2 or more, -2 or less	One side is too tall	No, needs a rotation

Worked example

Work from the bottom up. Leaves first, then their parents, and so on.



1. Nodes 10, 30 and 65 are leaves: height 0, BF 0.
2. Node 20 has two leaves below it: height 1, BF = 0 - 0 = 0.
3. Node 70 has a left child (height 0) and no right child (-1): height 1, BF = 0 - (-1) = **+1**.
4. Node 60 has no left child (-1) and a right child of height 1: BF = -1 - 1 = **-2**. This node is out of balance.
5. Node 40 has a left height of 1 and a right height of 2: BF = 1 - 2 = **-1**. That is fine.

Rotations

A rotation is a small change of pointers that makes the tall side shorter while keeping the BST order. After an insert or delete, go up from that spot towards the root. The first node with a BF of +2 or -2 is the one to fix. Call it z. Its taller child is y, and the taller child of y is x.

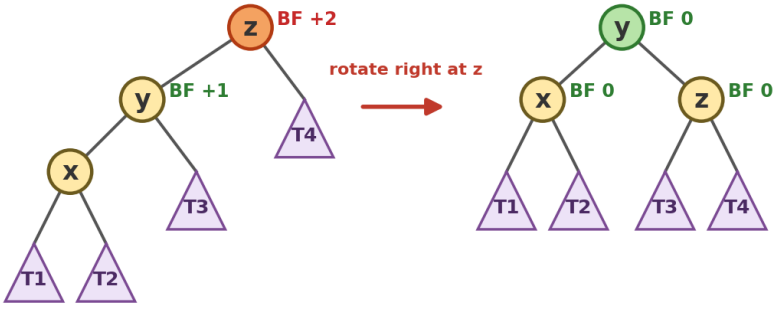
A simple way to choose: look at the path z, y, x. If it is a straight line, one rotation is enough. If it bends, you need two.

Case	Shape	What to do
LL	Straight line to the left (z is +2, y is +1)	Rotate right at z
RR	Straight line to the right (z is -2, y is -1)	Rotate left at z
LR	Bend: left, then right (z is +2, y is -1)	Rotate left at y, then rotate right at z
RL	Bend: right, then left (z is -2, y is +1)	Rotate right at y, then rotate left at z

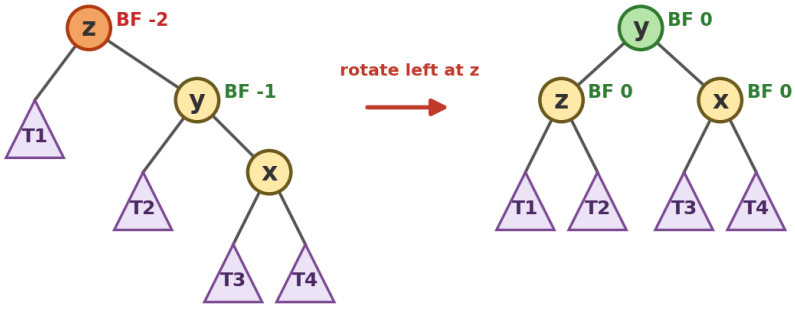
After a delete, y can have BF 0. Do not use two rotations in that case. One rotation is the right fix, same as LL or RR.

Nodes above z usually fix themselves once z is rotated, because its subtree gets back to the height it had before. Only rotate them if they are still out of balance after you recompute.

Case 1: Left-Left (LL) - one right rotation

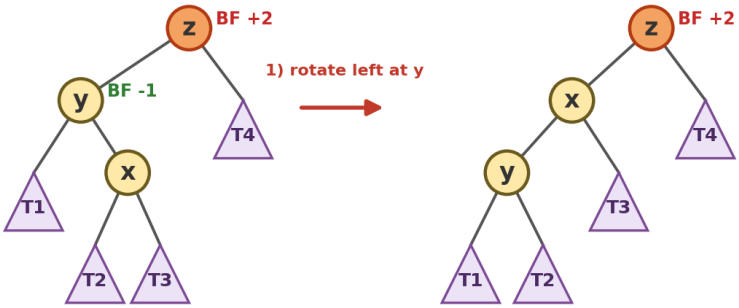


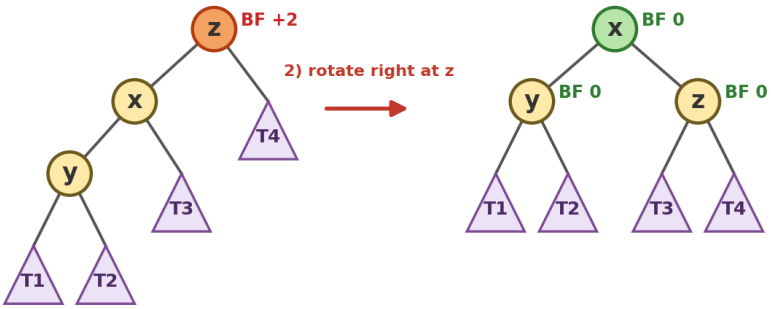
Case 2: Right-Right (RR) - one left rotation



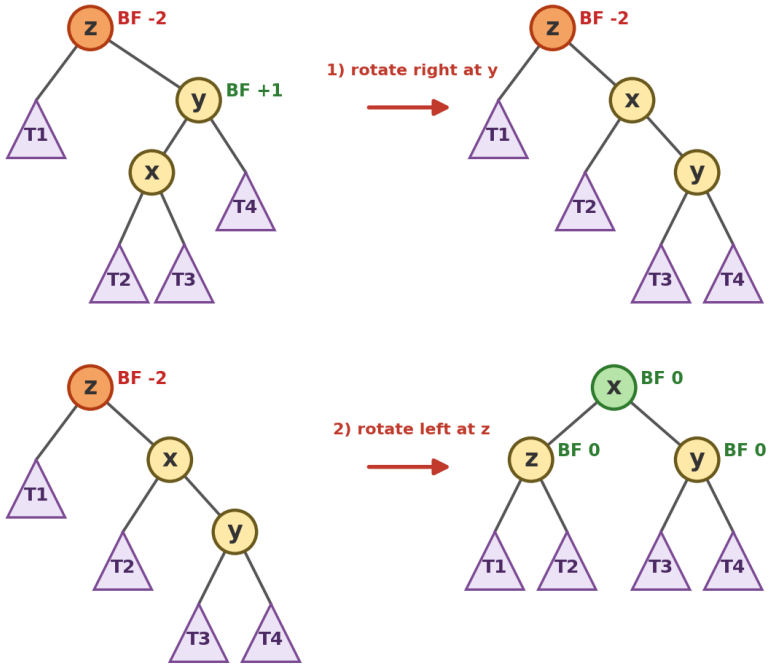
T1, T2, T3 and T4 are whole subtrees (they can be empty). They move together with their parent and stay in the same left-to-right order, so the BST rule never breaks.

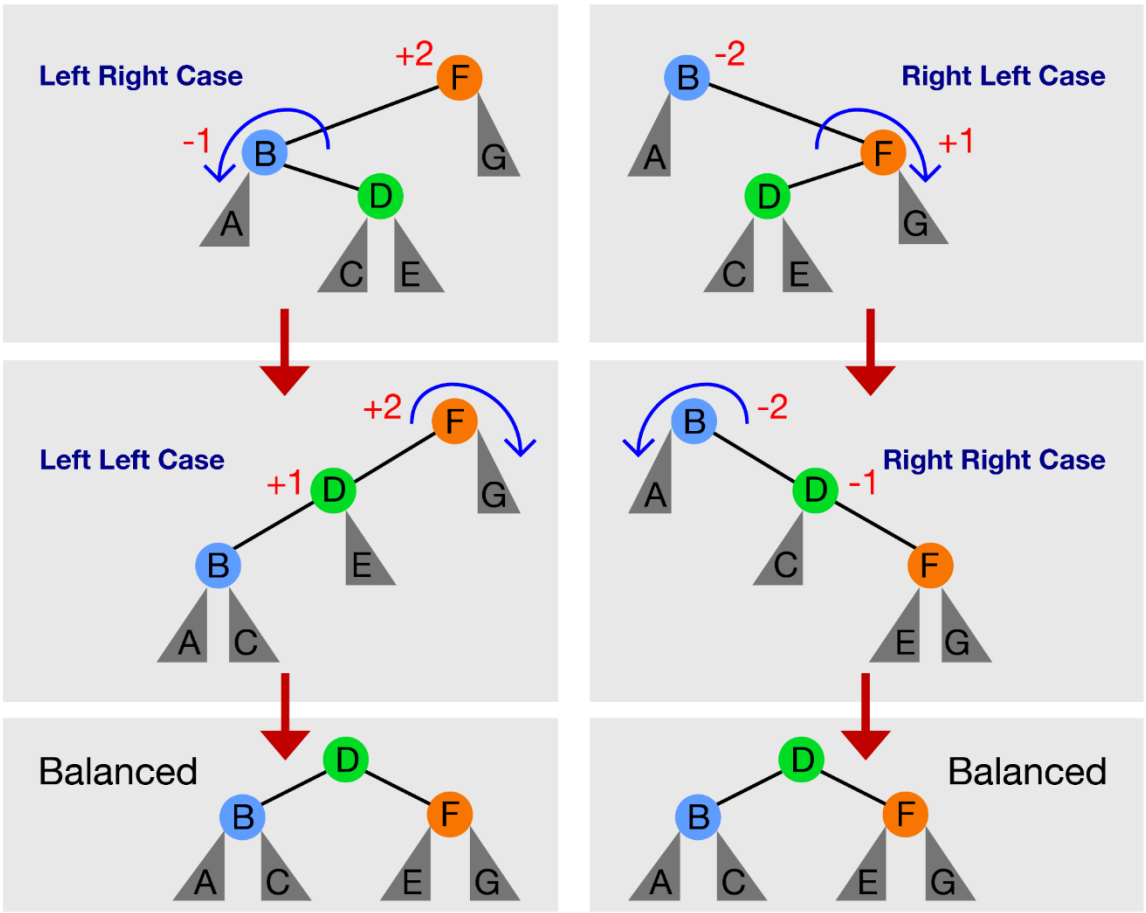
Case 3: Left-Right (LR) - step 1 then step 2





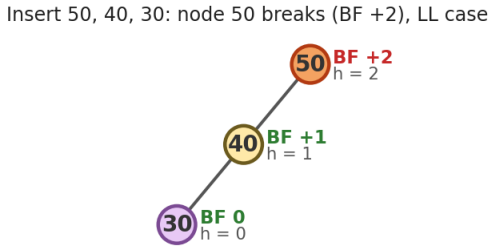
Case 4: Right-Left (RL) - step 1 then step 2

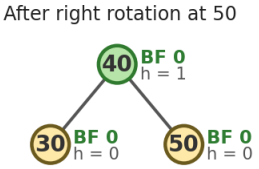


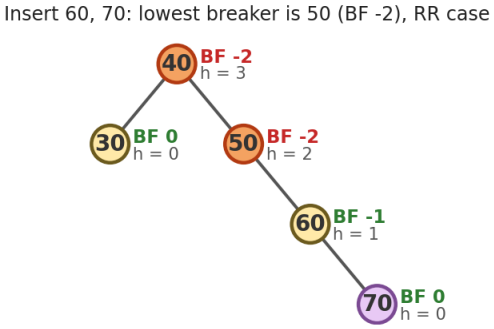


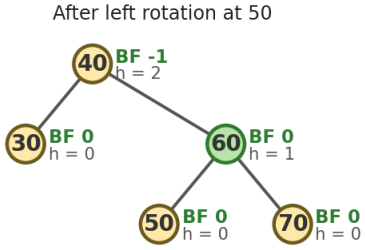
Example: inserting 50, 40, 30, 60, 70, 55

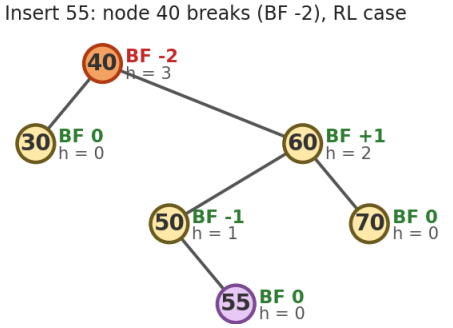
The left picture shows the tree right after the new key goes in. The right picture shows it after the fix.

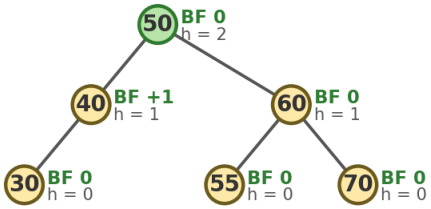












- After 30: node 50 has BF +2 and the path goes straight left, so it is **LL**. Rotate right at 50.
- After 70: node 50 has BF -2 and the path goes straight right, so it is **RR**. Rotate left at 50. Node 40 also shows -2, but we fix the lowest node first, and 40 is fine afterwards.
- After 55: node 40 has BF -2 and the path bends right, then left, so it is **RL**. Rotate right at 60, then left at 40.

BST vs AVL Tree: How They Compare

An AVL tree costs a little extra work on every insert and delete, and more code to write. In return, searching is fast every time.

	BST	**AVL Tree**
Search, insert, delete	O(n) in the worst case	O(log n) always
Height	Can grow up to n - 1	Stays close to log n
Staying balanced	Not handled	Automatic, using rotations
Best for	Simple storage	Lots of searching and updating



Background

In the world of Kung Fu Panda, Master Shifu wants the Scrolls of Kung Fu Warriors kept in such good order that he can find any warrior in a blink. He has asked you to build the Kung Fu Balance Tree: a templated AVL tree in C++.

Each warrior has a unique Warrior ID (the key the tree is ordered by), a Name and a Chi Power level. The tree balances itself after every insertion and deletion, just like Shifu teaches Po to keep his body and mind in balance.

With the tree, the masters can:

- **Add** new warriors
- **Remove** fallen ones
- **Search** for any fighter in O(log n) time

In-Lab Tasks

You can use the STL queue *and* stack *where a task asks for them. Everything else, including the tree, you write yourself.*

Task 1: Balance Factors and Rotations (30 min)

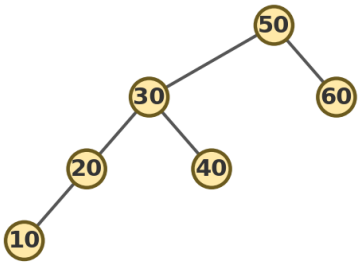
This task is on paper, no code.

1. Write the balance factor of every node. Remember that height counts edges, so start from the leaves.
2. Find the node that needs fixing. If more than one node is out of balance, pick the **lowest** one.
3. Say which case it is (LL, RR, LR or RL) and which rotation(s) you would do, and at which node(s).

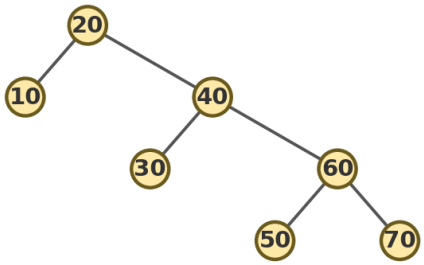
One tree in each set is already balanced. If you think a tree is fine, say so and show the balance factors that prove it. When you are done, show your answers to the lab instructor or TA before you start Task 2.

Task 1:

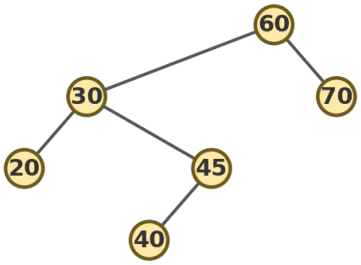
Tree 1



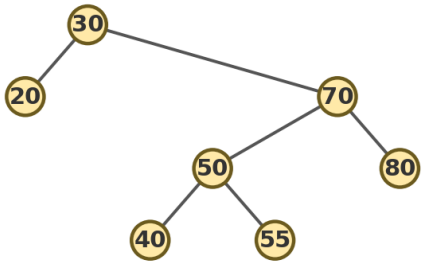
Tree 2



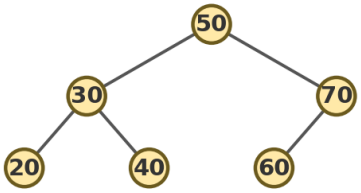
Tree 3



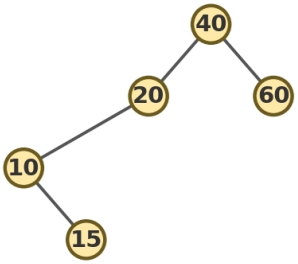
Tree 4



Tree 5



Tree 6



Task 2: The AVL Tree Class (55 min)

Write a templated AVL<T> class. Each node stores its data, its height, and pointers to its left and right child. Implement these:

1. insert(T data)
   - Insert like a normal BST. Ignore duplicates.
   - On the way back up, update heights and rebalance.
   - Every time you rotate, print one line such as Rotation: LR at 20 (the node the rotation was done at).
2. remove(T data)
   - Handle a leaf, a node with one child, and a node with two children (use the inorder successor).
   - Rebalance on the way back up and print the same Rotation: ... lines.
3. search(T data)
   - Return a pointer to the node, or nullptr if it is not there.
4. findMin() and findMax()
   - The leftmost and the rightmost key.
5. height(node), updateHeight(node), getBalanceFactor(node)
   - Height counts edges, so nullptr has height -1.
6. rotateRight(node) and rotateLeft(node)
   - The two single rotations. LR and RL are just these two used one after the other.
7. rebalance(node)
   - Checks the BF of the node and applies the right rotation(s) using the table from the theory.

Where to start

This is a skeleton and some pseudocode to get you going. It is not a finished solution, so write the real code yourself. Notice that insert and remove both finish by calling rebalance, so the rotation logic is written only once.

Node and class skeleton

```
template <typename T>
struct Node {
   T data;
   int height;
   Node* left;
   Node* right;
   Node(T val) : data(val), height(0), left(nullptr), right(nullptr) {}
};

template <typename T>
class AVL {
   Node<T>* root;
   int height(Node<T>* n) { return n ? n->height : -1; }
   Node<T>* insert(Node<T>* n, T data);   // returns the new root of this subtree
   Node<T>* remove(Node<T>* n, T data);
   Node<T>* rebalance(Node<T>* n);
public:
   AVL() : root(nullptr) {}
   void insert(T data) { root = insert(root, data); }
   void remove(T data) { root = remove(root, data); }
   // ... the rest of the functions
};
```

rebalance (the important one)

```
rebalance(n):
```

```
    updateHeight(n)
    bf = getBalanceFactor(n)
    if bf > 1:                          // left side too tall
       if getBalanceFactor(n->left) < 0:   // bend, so this is LR
          n->left = rotateLeft(n->left)
       return rotateRight(n)              // LL, or the second step of LR
    if bf < -1:                         // right side too tall
       if getBalanceFactor(n->right) > 0:  // bend, so this is RL
          n->right = rotateRight(n->right)
       return rotateLeft(n)               // RR, or the second step of RL
    return n                            // already balanced
```

Because the checks are < 0 *and* > 0*, a child with BF 0 (the delete case) goes to a single rotation automatically. Print your* Rotation: ... *line just before each* return rotate....

insert and remove

```
insert(n, data):
   if n is null: return new Node(data)
   if data < n->data:      n->left  = insert(n->left, data)
   else if data > n->data: n->right = insert(n->right, data)
   else: return n                       // duplicate
   return rebalance(n)

remove(n, data):
   if n is null: return null
   if data < n->data:      n->left  = remove(n->left, data)
   else if data > n->data: n->right = remove(n->right, data)
   else:
      if n has 0 or 1 child:
         child = (n->left) ? n->left : n->right
         delete n
         return child                   // may be null
      s = findMin(n->right)             // two children: inorder successor
      n->data = s->data
      n->right = remove(n->right, s->data)
   return rebalance(n)
```

A rotation, traced with numbers

```
rotateRight(y):
   x  = y->left
   T2 = x->right
   x->right = y
   y->left  = T2
   updateHeight(y)       // y is lower now, so update it first
   updateHeight(x)
   return x              // x is the new root of this subtree

// Trace: 50 -> 40 -> 30 going left. Call rotateRight(50).
// y = 50, x = 40, T2 = null.  Then x->right = 50 and y->left = null.
// height(50) = 1 + max(-1, -1) = 0
// height(40) = 1 + max(0, 0)   = 1
// Result: 40 with children 30 and 50. rotateLeft is the mirror image.
```

Check your work

Insert 50, 40, 30, 60, 70, 55 in this order (it is the example from the theory). Your program should print exactly these three lines:

Rotation: LL at 50
Rotation: RR at 50
Rotation: RL at 40

The final tree has 50 as the root, inorder 30 40 50 55 60 70 and preorder 50 40 30 60 55 70. Then try the trees from your Task 1 set: insert their keys in level order (top to bottom, left to right) first, then add the key you chose to break the balance, and check that your program picks the same case you wrote on paper.

Task 3: Printing and Traversing (30 min)

1. inorder(), preorder() and postorder(), written recursively
2. levelOrder(), using an STL queue
3. printTree(): print the tree sideways so you can see its shape. The root is on the left and the right subtree is on top. Show each node with its height and BF, with 4 spaces of indent per level. For the example tree it looks like this:

```
        70 [h0 bf0]
    60 [h1 bf0]
        55 [h0 bf0]
50 [h2 bf0]
    40 [h1 bf+1]
        30 [h0 bf0]
```

4. isBalanced(): returns true only if every node has a BF of -1, 0 or +1
5. clear(): frees every node safely

Where to start

```
printSideways(n, depth):
    if n is null: return
    printSideways(n->right, depth + 1)       // right first, so it is on top
    print (4 * depth) spaces, n->data, " [h", n->height, " bf", bf, "]"
    printSideways(n->left, depth + 1)

levelOrder():
    queue.push(root)
    while queue is not empty:
        cur = queue.front();  queue.pop()
        visit(cur)
        if cur->left:  queue.push(cur->left)
        if cur->right: queue.push(cur->right)
```

Print a + sign in front of positive BF values so your output matches the sample. Call printTree() *after every insert while you debug Task 2. It will save you a lot of time.*

Task 4: Kung Fu Balance Tree (25 min)

Create a Warrior class with an ID, a name and a chi power. Order the tree by ID, so give Warrior the comparison operators (< and >) and << for printing. Then build a menu with these options:

1. Add a warrior (show a message if the ID already exists)
2. Remove a warrior by ID
3. Find a warrior by ID and show the details
4. List all warriors in ID order
5. Show the strongest and the weakest warrior (highest and lowest chi power)
6. Show tree stats: number of warriors, height, and whether the tree is balanced
7. Show the tree sideways using printTree()

The tree is ordered by ID and not by chi power, so for the strongest and weakest warrior you have to visit every node and remember the best one so far.

Bonus: list only the warriors whose ID lies between two numbers, without visiting subtrees that cannot contain any.

```
listInRange(n, lo, hi):
  if n is null: return
  if lo < n->id: listInRange(n->left, lo, hi)
  if lo <= n->id and n->id <= hi: print n
  if n->id < hi: listInRange(n->right, lo, hi)
```

Test with at least 10 warriors, including some whose IDs arrive in increasing order, and show that the tree stays balanced.

Submission Guidelines

1. Submit **all** files, named with your roll number: l241234.cpp
2. Show your Task 1 answers to the lab instructor or TA before you leave.